

AI Master Project 3기

최종 발표

과제명 Mini AI-Ops

멘 티 신희승, 10068

멘 토 이재영

발표 영상 10분 이내 | 시연 영상 별도 제출 5분 이내

프로젝트 개요

권장 2분 이내

1. 문제 정의 (Vision / Problem)

어떤 문제를 해결하고자 했는가?

배포 후 및 정기 점검마다 운영자가 대량의 로그를 직접 확인하는 반복 업무를 수행하는데 이는 길게는 30분 이상 소요되고 있으며, 운영자가 체득하고 있는 정보가 지식화 되지 않고 있습니다.

반복되는 작업을 줄이고, 이 결과를 지식화 하고자 합니다.

왜 이 문제가 중요한가? (비즈니스 임팩트)

로그 점검 자동화로 배포 후 점검 시간 줄여, 반복 점검에 투입되는 휴먼 리소스를 절감하고, 확보된 인력을 장애 대응과 서비스 품질 개선 등 고부가가치 업무에 집중시킬 수 있습니다. 또한 인력이 변경되어도 기존 지식을 바로 승계 할 수 있습니다.

2. 핵심 기능 (Core Features)

무엇을 만들었는가?

로그 수집·정규화, 패턴 분류, 이상 탐지, RAG 기반 대응 권고 및 운영 지식 축적을 자동화한 Mini-AIOps 시스템을 구축했습니다.

에이전트 구성:

OrchestratorAgent(작업 조율) → LogCollectorAgent(로그 수집) → LogAnalysisAgent(정규화·패턴 분석) → AnomalyDetectionAgent(이상 탐지) → KnowledgeBaseRAGAgent(유사 사례 검색) → RecommendationAgent(대응 방안 생성)

핵심 기술 스택:

FastAPI · LangGraph · MCP-style Tool Registry · SQLite · ChromaDB · RAG · OpenAI/Azure OpenAI Embeddings · Vue 3 · TypeScript

핵심 성과 (Key Achievements)

지식축적

운영자 개인에게만 쌓이는 지식을
재사용 가능한 운영 지식으로 전환

점검시간

로그 전수 조사에 걸리는 시간 감소
약 30분 > 5분(추정)

Pattern 인식

신규패턴 인식을 90% 이상
기존패턴 인식을 80% 이상

Key Message

“로그 점검 자동화로 반복 업무 시간을 70% 단축하고, 검증된 분석
조치 결과를 재사용 가능한 운영 지식으로 전환했습니다.”



기술 아키텍처 고려 사항

아키텍처 구성에 대한 핵심 결정 사항을 중심으로 설명하세요

01 Deterministic 로그 정규화 + PatternOps Registry

선택 이유:

로그의 변동값으로 동일 장애가 여러 패턴으로 분리되는 문제를 방지하고, 검증된 패턴과 대응 기준을 지속적으로 재사용하기 위해 적용했습니다.

핵심 활용:

로그를 정규화해 Fingerprint로 그룹화하고, PatternOps Contract와 비교하여 Known/New Pattern을 판정하고 원인·조치 근거를 연결했습니다.

02 Knowledge Card RAG + Recommendation Quality Gate

선택 이유:

모델 재학습 없이 축적된 장애 사례를 활용하고, LLM의 근거 없는 답변이나 위험한 운영 권고를 통제하기 위해 적용했습니다.

핵심 활용:

ChromaDB에서 유사 Knowledge Card를 검색해 권고 근거로 활용하고, 생성 결과를 품질 점수와 Hard-fail 기준으로 검증해 재생성 또는 안전한 Fallback을 수행했습니다.

03 LangGraph 기반 Multi-Agent Orchestration

선택 이유:

단순 Chain으로 관리하기 어려운 에이전트별 조건부 실행, 상태 기반 분기, 재시도 및 실패 이력을 체계적으로 제어하기 위해 적용했습니다.

핵심 활용:

Orchestrator가 로그 수집·패턴 분석·이상 탐지 에이전트를 조율하고 실행 결과와 근거를 SharedState에 누적했으며, 상세 권고는 선택된 Fingerprint에 대해 별도 Workflow로 실행했습니다.

핵심 기술 난제 (Hurdle)

단순 구현으로 해결되지 않았던 핵심 기술 과제를 제시하세요.

request ID·경로·숫자처럼 계속 변하는 값 때문에 동일 장애가 서로 다른 Fingerprint로 분산되며, 고정 정규식만으로는 새 변형을 지속 흡수하기 어렵다. 따라서 정규화를 일회성 전처리가 아니라, 운영자 검증 결과가 다음 분석의 판정 기준으로 환류되는 장기 운영 학습 레이어로 전환해야 했다.

엔지니어링 접근 방법

어떻게 해결했는가? (설계 결정)

정규화와 Fingerprint를 1차 기준으로 두고, Drain3·hybrid similarity·HDBSCAN으로 중복 후보를 만든 뒤 운영자 승인 시 normalization rule, canonical Fingerprint alias, Known Pattern을 함께 저장하도록 설계하여 “Human-in-the-loop 기반 지속 학습형 운영 메모리”를 사용한다.

핵심 알고리즘 / 아키텍처 결정

정규화 후 Drain3·hybrid similarity·HDBSCAN으로 중복 후보를 만들고, 승인된 rule·alias·Known Pattern을 저장했다.

LangGraph와 PatternOps가 이를 다음 분석에 재적용하는 “Human-in-the-loop 운영 메모리”를 구성했다.

왜 이 접근이 필요했는가?

정규식만으로는 새 변형을 놓치고, embedding만으로는 구조가 다른 로그를 잘못 합칠 수 있다. 결정적 규칙·다중 근거·운영자 승인을 결합해 재현성과 오탐 통제를 확보했다.

결과 및 성과 (수치로 제시)

98%

성과 지표 1 – FingerPrint 분산 98%축소

- 중복으로 처리되는 패턴 50건 > 1건으로 처리

100%

성과 지표 2 - 규칙 재사용률

- Known Pattern 변형 50건을 50/50(100%) Known Pattern으로 흡수

4배

성과 지표 3 - 분석 권고 품질 개선

- 종합 품질: 87.1→95.6점, 9.8% 향상
- 검증 구체성: 11.0→13.5점, 22.7% 향상
> 90점 이상 고품질 권고 생성 비율 4배 증가